UNIVERSIDAD SERGIO ARBOLEDA

Escuela de Ciencias Exactas e Ingeniería

Ingeniería en Ciencias de la Computación e Inteligencia Artificial (ICCIA)

Informe de Avance Técnico y Análisis Big O Preliminar

Entrega Parcial (Sesión 3) - Entregable 3

Paradigmas de Programación (SIST0176) - Semestre III

Grupo 4: Segmentación y Procesamiento de Imágenes Médicas por Visión Artificial

Integrantes: [Nombre 1], [Nombre 2], [Nombre 3], [Nombre 4] (completar)

Docente: [completar] | Fecha: noviembre de 2026

1. Estado de avance

Módulo / actividad	Estado a la Sesión 3	Evidencia
Módulo 1 - Kernel imperativo C++17	Terminado: convolución 2D (Gauss 5x5, Sobel 3x3), padding contiguo, OpenMP por filas	cpp/, build/convolve, build/bench
Módulo 3 - Pipeline funcional Python	Terminado: normalización, máscara, recursión de cola con trampolín, Option, currificación	funcional/, 12 pruebas unitarias OK
Módulo 2 - POO Java 17	Terminado e integrado: dominio inmutable, Factory Method, DIP, servicio con lote concurrente	java/src, 18 pruebas OK
Integración Java -> Python -> C++ -> Python	Funcionando de punta a punta con datos sintéticos	make demo
Pruebas de inmutabilidad y aliasing	Hechas (7 pruebas de copias defensivas e inmodificabilidad)	tac.pruebas.PruebasTac
Benchmarks Secuencial vs Paralelo	Programa listo; resultados de esta máquina en el informe final	cpp/bench.cpp, docs/benchmarks.md
Pendiente (Sesión 4-5)	Refactor final, benchmarks en el equipo del grupo, informe final, diapositivas y sustentación	-

Datos: las TAC son **sintéticas** (fantoma generado con datos/generar_tac.py: cráneo, cerebro, ventrículos, tumores, ruido y desenfoque). No son imágenes de pacientes ni validan un uso clínico.

2. Notación y supuestos

W = ancho, **H** = alto (en píxeles), **K** = lado del kernel (impar), **r = piso(K/2)**, **p** = número de hilos, **M** = cantidad de píxeles marcados en la máscara de segmentación (M ≤ W·H), $\mathbf{n_r}$ = píxeles de una región. Se cuenta como operación elemental cada multiplicación, suma, comparación o acceso a memoria; su costo se acota por una constante **c**.

3. Módulo 1 (C++): convolución 2D

3.1 Complejidad temporal

Para cada píxel de salida (y, x) se calcula:

$$out(y, x) = \Sigma_{i=0}^{K-1} \Sigma_{j=0}^{K-1} w(i, j) \cdot pad(y + i, x + j)$$

Cada término cuesta una multiplicación, una suma y dos lecturas: a lo sumo **c** operaciones. Entonces:

$$T(W, H, K) = \Sigma_{y=0}^{H-1} \Sigma_{x=0}^{W-1} \Sigma_{i=0}^{K-1} \Sigma_{j=0}^{K-1} c = c \cdot H \cdot W \cdot K \cdot K = c \cdot W \cdot H \cdot K^2$$

Como el costo por término es constante, existen $c_1 = c_2 = c$ tales que $c_1 \cdot W \cdot H \cdot K^2 \le T \le c_2 \cdot W \cdot H \cdot K^2$; por definición **T(W, H, K) = Θ(W·H·K²)**.

Costo del padding. Se copia una imagen de tamaño (W + K - 1)(H + K - 1) = W·H + (K-1)(W + H) + (K-1)². Como W + H ≤ 2·W·H, se cumple (K-1)(W+H) ≤ 2·W·H·K², y además (K-1)² ≤ W·H·K². Luego el padding es O(W·H·K²) y no cambia el orden total: $\mathbf{T_{total}}$ **= Θ(W·H·K²)**.

3.2 Complejidad espacial

$$S(W, H, K) = (W + K - 1)(H + K - 1)\ [pad] + W \cdot H\ [salida] + K^2\ [kernel] = \Theta(W \cdot H + K(W + H) + K^2)$$

Si K ≤ min(W, H) (siempre cierto en imágenes reales) entonces K·W ≤ W·H y K·H ≤ W·H, por lo que K(W + H) ≤ 2·W·H y K² ≤ W·H. Resulta **S = Θ(W·H)** (espacio auxiliar, además de la entrada).

3.3 Detección de bordes (Gauss 5x5 + Sobel X + Sobel Y + magnitud)

$$T_{bordes} = c \cdot W \cdot H \cdot (5^2 + 3^2 + 3^2) + c' \cdot W \cdot H = c \cdot W \cdot H \cdot 43 + c' \cdot W \cdot H = \Theta(W \cdot H)$$

Con K fijos (5 y 3) los factores K² son constantes: el pipeline de bordes es **lineal en el número de píxeles**. En espacio se usan 5 arreglos auxiliares de W·H floats (liso, gx, gy, magnitud, pad) y la copia de entrada: **Θ(W·H)**. Para 256×256 son unos 1.6 MB.

3.4 Versión paralela (OpenMP)

Corrección (sin condiciones de carrera). La iteración y escribe únicamente en la fila y de la salida: los conjuntos de escritura {out[y·W .. y·W + W - 1]} son disjuntos para y distintos, y los arreglos pad y kernel solo se leen. Se cumplen las condiciones de Bernstein (ningún par de iteraciones tiene conflicto escritura/escritura ni lectura/escritura), así que no hay *data races*. No se usan locks, de modo que tampoco hay *deadlocks*; la única sincronización es la barrera implícita al final del `parallel for`. El benchmark comprueba además que el resultado paralelo es **idéntico bit a bit** al secuencial.

Tiempo. Con planificación estática, cada hilo procesa a lo sumo techo(H/p) filas de costo Θ(W·K²); el padding se mantiene secuencial:

$$T_p = \Theta(techo(H/p) \cdot W \cdot K^2) + \Theta(W \cdot H) = \Theta(W \cdot H \cdot K^2 / p + W \cdot H)\ (p \le H)$$

Aplicando la ley de Amdahl con fracción secuencial f ≈ 1/(1 + K²) (una unidad de copia por píxel frente a K² de cálculo):

$$S_{máx}(p) = 1 / (f + (1 - f)/p)$$

Kernel	f (secuencial)	p = 2	p = 4	p = 8
K = 3	10.0%	1.82x	3.08x	4.71x
K = 5	3.8%	1.93x	3.59x	6.30x
K = 9	1.2%	1.98x	3.86x	7.37x

Estos valores son un modelo teórico (cota superior), no mediciones: ignoran el ancho de banda de memoria y el costo de crear hilos.

3.5 Localidad de caché

La imagen se guarda por filas (row-major) en un bloque contiguo de memoria. El bucle interno recorre K floats consecutivos de cada fila (`fp[kx]`), que caben en una o dos líneas de caché de 64 bytes (16 floats); al avanzar x se reutilizan K-1 de ellos. El padding con bordes replicados elimina los `if` de borde del bucle interno, y paralelizar por filas evita que dos hilos escriban en la misma línea de caché (salvo en el borde entre bloques).

4. Módulo 3 (Python funcional)

Función	Tiempo	Espacio	Justificación
normalizar (map + ventana)	$\Theta(W \cdot H)$	$\Theta(W \cdot H)$	Una aplicación O(1) por píxel; se crea una tupla nueva.
mascara (map sobre 2 tuplas)	$\Theta(W \cdot H)$	$\Theta(W \cdot H)$	Una comparación de banda y una de borde por píxel.
componentes (recursión de cola)	$O(M^2)$ peor caso	$O(M)$	Ver análisis abajo.
estadisticas (sorted + reduce)	$\Theta(n_r \log n_r)$	$O(n_r)$	sorted es n log n; reduce recorre n elementos una vez.
region_mayor (reduce)	$\Theta(R)$	$O(1)$	Un paso por región; R = cantidad de regiones.
segmentar (total)	$O(W \cdot H + M^2)$	$O(W \cdot H)$	Suma de las anteriores; domina M^2 solo si M es grande.

Análisis de componentes()

En `_crecer` cada paso extrae un elemento de la frontera. Un píxel entra a la frontera como máximo 4 veces (una por vecino), así que los pasos de una región con n_r píxeles son $\leq 4 \cdot n_r + 1$. Cada paso hace `region | {actual}`, que copia un frozenset de hasta n_r elementos (inmutabilidad), y `frontera[1:]`, que copia la frontera ($\leq 4 \cdot n_r$). La parte dominante es:

$$\Sigma_{k=1}^{n_r} k = n_r(n_r + 1)/2 = \Theta(n_r^2) \text{ por región}$$

Sumando sobre todas las regiones, $\Sigma\, n_r^2 \leq M^2$; además `min(pendientes)` y `pendientes - region` cuestan O(M) por región, o sea $O(R \cdot M) \leq O(M^2)$. Total: **$O(M^2)$** en el peor caso. En las imágenes de prueba $M \approx 600$ píxeles, muy por debajo de $W \cdot H = 65\,536$.

Costo de la pureza. Una versión con conjunto mutable (BFS) sería O(M); aquí se acepta $O(M^2)$ porque el requisito es inmutabilidad y transparencia referencial. Se podría recuperar O(M log M) con estructuras de datos persistentes (por ejemplo, HAMT).

Pila. Python no optimiza llamadas de cola. El trampolín ejecuta cada paso en un bucle, por lo que la profundidad de pila es **O(1)** frente a $O(n_r)$ de la recursión directa (la prueba `test_recursion_de_cola_no_desborda_la_pila` recorre 20 000 pasos, más que el límite de

recursión).

5. Módulo 2 (Java) e integración

Operación	Tiempo	Espacio	Comentario
EstudioMedico (constructor) y getPixeles()	$\Theta(W \cdot H)$	$\Theta(W \cdot H)$	Copia defensiva con clone(): el precio de evitar aliasing.
Leer / escribir PGM (FormatoPGM)	$\Theta(W \cdot H)$	$\Theta(W \cdot H)$	Un token por píxel.
ResultadoAnalisis (constructor)	$\Theta(W \cdot H + R)$	$\Theta(W \cdot H + R)$	Clona 2 arreglos y copia la lista de R regiones.
ServicioDiagnostico.analizar	$\Theta(W \cdot H \cdot (K_g^2 + K_s^2)) + O(M^2)$	$\Theta(W \cdot H)$	Suma de C++ ($\Theta(W \cdot H)$ con K fijo) y segmentación Python.
analizarLote con p hilos y N estudios	$\approx (N / p) \cdot T_{estudio}$	$\Theta(p \cdot W \cdot H)$	Cada tarea usa archivos temporales propios; resultados inmutables.

6. Plan para las sesiones 4 y 5

- Medir speedup real en el computador del grupo con `make bench` y completar las tablas del informe final.
- Refinar la separación Functional Core / Imperative Shell y documentar los efectos secundarios.
- Preparar diapositivas (máx. 10) y repasar el código para el interrogatorio individual.